Конспект: Архитектура вычислительных систем

Oct 29, 2026 · @Sokrat

Как устроен конспект

Курс разбирает корпоративную ИТ-инфраструктуру как стек слоёв: от сети и дисков внизу до приложений наверху. Каждая лекция берёт один слой и разбирает его подробно, поэтому конспект идёт в том же порядке.

Лектор: Артём Береснев (ИТМО). Курс вводный: цель — целостная картина технологического стека с упором на контекст ИИ, практика на Linux. Разработка ПО, управление разработкой и ITSM в курс не входят.

Раздел	Слой стека	Главный вопрос
Лекция 1. Введение	весь стек	Из каких слоёв состоит инфраструктура и зачем каждый нужен
Лекция 2. Слой Servers	Servers	Из чего состоит вычислительная платформа: CPU, GPU, плата, память
Лекция 5, ч. 1. Storage	Storage (железо)	Как устроены диски, интерфейсы и дисковые протоколы
Лекция 5, ч. 2. Storage	Storage (системы)	DAS / NAS / SAN, RAID, LVM, измерение скорости
Глоссарий	—	Все аббревиатуры курса в одном месте

В тексте помечено, что взято со слайдов, а что добавлено для понимания: блоки «Зачем это / как понять» — пояснения сверх слайдов. Цифры и примеры оборудования — со слайдов, если не сказано иное.

Лекция 1. Введение: ИТ-инфраструктура как стек слоёв

Главная идея лекции: любую информационную систему можно разложить на 9 слоёв — от сети до приложения. Каждый слой пользуется сервисами нижнего и предоставляет свои верхнему. Модель взята из Cloud Computing Reference Architecture (NIST).

Зачем инфраструктура специалисту по ИИ

Лектор приводит три истории, и в каждой проблема была не в модели, а в инфраструктуре:

1. **Утечка данных (2020).** Из-за неправильно настроенного ELK (Elasticsearch + Logstash + Kibana) в сеть утекли данные около 150 млн пользователей соцсетей. Это ошибка слоёв Data/Middleware и безопасности, а не кода.
2. **ML-стартап и медленный диск.** Для обучения на ~12 ТБ фотографий взяли машину с NVIDIA A100, Xeon и 256 ГБ ОЗУ, но на одном HDD на 15 ТБ. Видеокарты простаивали, потому что диск не успевал отдавать данные. Помог tiering — связка быстрого NVMe SSD и ёмкого HDD. Итог: сорванные сроки и почти 100 тыс. впустую.
3. **AI-стартап по защите блокчейнов.** У клиентов «зоопарк технологий», поэтому ML-инженеру приходится постоянно думать об архитектуре. Нужны навыки software engineer, а не только ML.

Вывод лектора: будут не профессии, а роли; всё, что можно алгоритмизировать, алгоритмизируют. Ценность университетского образования — фундамент, широкий кругозор и умение быстро разбираться в незнакомом.

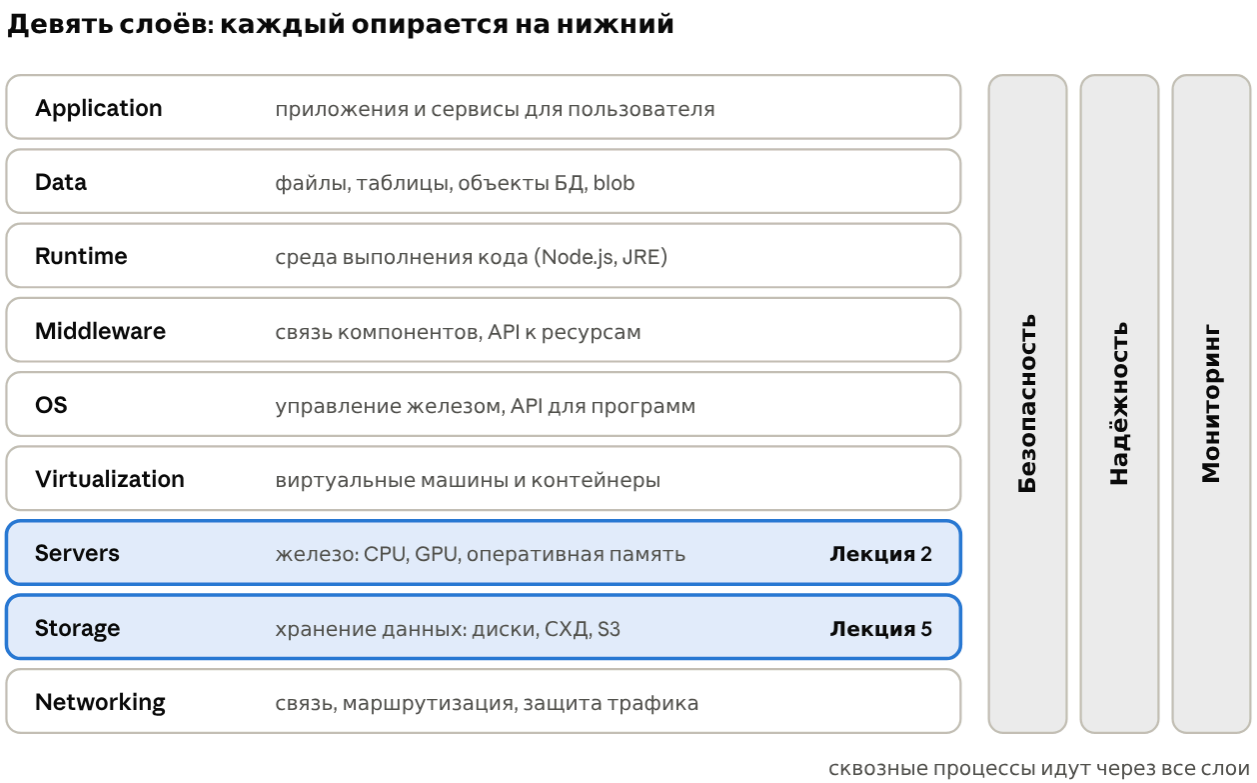
Три понятия, которые легко спутать

Понятие	Определение (со слайдов)	Как понять
Информационная система (ИС)	Организационно упорядоченная совокупность документов и информационных технологий, реализующих информационные процессы: хранение, обработку, поиск, передачу информации	Работающая система целиком: данные + технологии + процессы, ради которых она создана. Пример: CRM компании
Технологический стек	Набор технологий, языков, библиотек, фреймворков и инструментов для разработки и запуска ПО	«Из чего сделано конкретное приложение»: Python + Flask + React + Postgres. Описывает одно приложение

Понятие	Определение (со слайдов)	Как понять
Корпоративная ИТ-инфраструктура	Комплекс аппаратных и программных ресурсов, сетевых компонентов и сервисов, которые поддерживают информационные процессы и автоматизируют бизнес организации	«На чём живут все приложения компании»: серверы, СХД, сеть, виртуализация, ОС. Общая для многих систем

Иначе говоря: стек — взгляд разработчика одного продукта, инфраструктура — взгляд компании на фундамент под всеми продуктами, а ИС — всё вместе плюс люди и процессы.

Девять слоёв инфраструктуры



стек ИТ-инфраструктуры · 9 слоёв и 3 сквозных процесса

Курс дальше разбирает слои по одному: Servers — в лекции 2, Storage — в лекции 5.

Слой	Что делает	Примеры со слайдов
Application	Пользовательские приложения и сервисы (desktop, web, мобильные, серверные), дают конечному пользователю функциональность	Яндекс.Маркет, Trello, MS Office, AI-агенты
Data	Хранение, управление и обработка структурированных и неструктурированных данных: файлы, таблицы, объекты БД, blob	Векторные БД (Qdrant, Chroma), реляционные (Postgres, MS SQL), MongoDB, Google BigQuery
Runtime	Среда выполнения: интерпретаторы и контексты, доступные программе во время работы	Node.js, JRE, PHP, .NET / .NET Core, CUDA Runtime API
Middleware	Промежуточное ПО: API между приложением и нужными ему ресурсами; связь между компонентами	CUDA, RPC, ODBC, JDBC; брокеры сообщений (MOM) Kafka, RabbitMQ, Azure Service Bus; иногда СУБД
OS	Управляет аппаратными ресурсами и даёт программам сервисы через API	Ubuntu, Windows Server, RHEL, Android
Virtualization	Абстрагирует физические ресурсы и отдаёт их как виртуальные машины или контейнеры	VMware vSphere, Hyper-V, KVM, VirtualBox, Parallels, Docker / Kubernetes
Servers	Физическое железо, дающее вычислительные ресурсы (CPU, GPU, RAM)	Серверные платформы — подробно в лекции 2
Storage	Хранение данных и доступ к ним для приложений и служб	DAS / NAS / SAN, SDS, Amazon S3, Google Cloud Storage, NetApp
Networking	Соединение, маршрутизация и безопасность передачи данных	Коммутаторы, маршрутизаторы, стеки протоколов, SDN

Зачем это / как понять. Стек нужен, чтобы локализовать проблему и распределить ответственность. В истории со стартапом «медленно учится модель» виноват был слой Storage, хотя симптом был виден на слое Application.

Близкая аналогия — модель OSI в сетях: каждый уровень решает свою задачу и не думает о деталях соседних.

Runtime и Middleware: в чём разница

Эти два слоя чаще всего путают. **Runtime** — то, *в чём* исполняется код: без Node.js файл `server.js` — просто текст. **Middleware** — то, *через что* код общается с другими компонентами: с БД через JDBC/ODBC, с другим сервисом через Kafka, с клиентами через сервер приложений.

Пример со слайда — минимальный веб-сервер на Node.js:

```
const http = require('http');          // модуль стандартной библиотеки
Node.js
const hostname = '127.0.0.1';          // сервер работает локально
const port = 3000;
const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello World\n');             // одинаковый ответ на любой запрос
});
server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

Разложение по слоям: сам код — Application; Node.js, который его интерпретирует, — Runtime; сетевой стек, на который опирается `listen()`, предоставляет OS.

Пример: стек веб-приложения для управления проектами

Слой	Что используется
Application	Бэкенд на Python + Flask, фронтенд на React
Data	БД из 47 таблиц с хранимыми процедурами и представлениями (views) в Postgres
Runtime	Интерпретатор Python и библиотеки Flask
Middleware	Сервер приложений Gunicorn + кэш Redis
OS	Debian 12: один экземпляр под Gunicorn, второй под Redis

Слой	Что используется
Virtualization	Виртуальные машины у облачного провайдера
Servers	Железо DEPO
Storage	Локальные диски серверов + S3-хранилище для файлов
Networking	Трафик идёт через инфраструктуру провайдера с DNAT и защитой от атак через ddos-guard

Полезное упражнение: разложить так же свой проект. Например, в ЛР 2 есть Docker (Virtualization), Python-сервисы (Application + Runtime) и SSH-доступ между узлами (Networking).

Кто управляет слоями в облаке (дополнение)

На слайдах этого нет, но именно для этого стек и задуман в модели NIST. Облачные модели отличаются тем, до какого слоя всё делает провайдер.

Модель	Провайдер управляет	Вы управляете	Пример
On-premise	ничем	всеми 9 слоями	Свой серверный зал
IaaS	Networking, Storage, Servers, Virtualization	OS и всем выше	Виртуальная машина в облаке
PaaS	всем до Runtime включительно	Data и Application	Платформа, куда заливаешь только код
SaaS	всеми слоями	только своими данными и настройками	Trello, MS Office 365

Сквозные процессы: что идёт через все слои

Есть задачи, которые нельзя решить на одном слое: они реализуются на всех уровнях сразу.

- **Безопасность** — непрерывный процесс обеспечения требуемого уровня конфиденциальности, целостности и доступности (триада CIA). Утечка из ELK — нарушение конфиденциальности.
- **Надёжность** — непрерывный процесс обеспечения требуемого уровня доступности сервиса. Примеры на разных слоях: RAID на Storage, несколько блоков питания на Servers, репликация на Data.
- **Мониторинг** — непрерывное наблюдение, оценка и анализ работы ИТ: серверов, сетей, БД, приложений. Цель — надёжность, эффективность и безопасность.

Выводы лекции 1

- Архитектуру ИТ-инфраструктуры можно представить стеком, хоть и условно; для ИТ это типично.
- Не все слои обязательны. Например, приложение может работать на железе без виртуализации (bare metal), а у простого скрипта нет Middleware.
- Безопасность, надёжность и мониторинг — не слои, а процессы поверх всех слоёв.

Лекция 2. Слой Servers: из чего состоит вычислительная платформа

Сервер — это вычислитель (CPU и/или GPU), память и ввод-вывод, соединённые шинами на материнской плате. Производительность определяется самым медленным звеном этой цепочки, поэтому важно понимать каждый компонент.

Немного истории

Год	Событие	Почему важно
1945	Джон фон Нейман описал архитектуру программируемого компьютера	Программу можно менять, не меняя железо
1947	Изобретение транзистора	Базовый элемент всей современной электроники
середина 1960-х	Большие интегральные схемы (БИС)	Много транзисторов на одном кристалле
начало 1970-х	Микропроцессор	Весь вычислитель в одной микросхеме

Год	Событие	Почему важно
1980-е	Принцип открытой архитектуры IBM PC	Любой производитель может делать совместимые модули — отсюда модульные системы

Любая платформа состоит из пяти частей: **вычислитель, ОЗУ, ПЗУ, ввод/вывод и шины передачи данных**. ОЗУ — быстрая временная память, стирается при выключении. ПЗУ — постоянная, там живёт прошивка (BIOS/UEFI).

Принстонская и гарвардская архитектуры

Разница — в том, лежат ли команды программы и данные в одной памяти.

	Принстонская (фон Неймана)	Гарвардская
Устройство	Одна память и общая шина для команд и данных	Память программ и память данных физически разделены
Плюсы	Эффективное использование памяти, программа может менять команды как данные	Быстродействие: команда и данные читаются одновременно
Минусы	Производительность упирается в шину («бутылочное горлышко фон Неймана»)	Несколько шин, ограничения на размер каждой памяти
Где сегодня	Внешняя структура почти всех процессоров и платформ: одна ОЗУ	Внутри процессора: отдельные кэши команд (L1I) и данных (L1D)

Как понять: современный процессор — гибрид. Снаружи он фон-неймановский (одна ОЗУ на всё), а внутри, на уровне кэша L1, гарвардский.

Модульная система и SoC

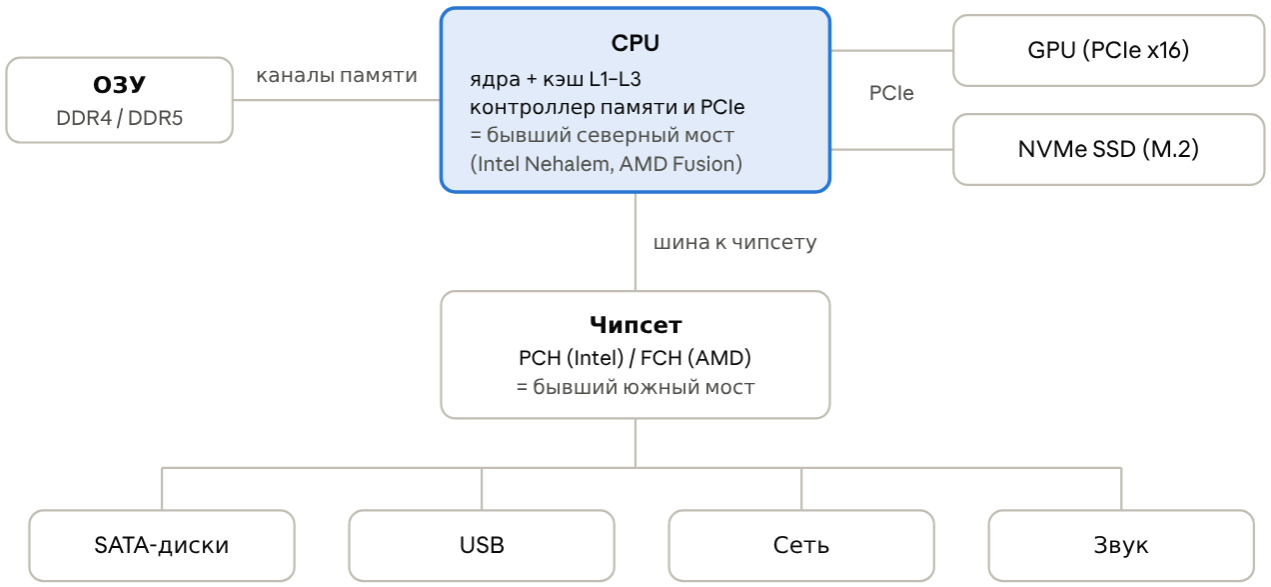
Модульная система (наследник IBM PC) собирается из сменных блоков: CPU (может быть несколько), материнская плата, память, диски, блоки питания, видеокарты, сетевые интерфейсы, контроллеры дисков, корпус. В центре — материнская плата с шинами, ПЗУ, цепями питания и тактовыми генераторами.

- **Северный мост (Northbridge)** — раньше отдельный чип для быстрых устройств: ОЗУ и графики (AGP, PCI Express). Сейчас встроен в сам процессор: у Intel с архитектуры Nehalem, у AMD с Fusion (там в процессор переехал и GPU).

- **Южный мост (Southbridge)** — чип для медленной периферии: SATA, USB, звук, сеть, встроенные контроллеры. У Intel называется PCH (Platform Controller Hub), у AMD — FCH (Fusion Controller Hub). Сегодня это и есть «чипсет».

Зачем северный мост перенесли в CPU. Каждый лишний чип на пути к памяти добавляет задержку. Когда контроллер памяти и PCIe внутри процессора, путь короче и быстрее.

Быстрые устройства — к CPU напрямую, медленные — через чипсет



современная модульная платформа · CPU, память, PCIe, чипсет

SoC (System on a Chip) — доведённая до конца интеграция: процессорные ядра, графика, контроллеры памяти и периферии на одном кристалле. Примеры: Apple M-серии, NVIDIA DGX Spark. Плюс — компактность и энергоэффективность; минус — ничего нельзя заменить по отдельности.

Три смысла слова «архитектура процессора»

1. **Подход к разработке:** CISC, RISC, MISC, VLIW.
2. **Набор команд (ISA, «ассемблер»):** x86-64 (он же AMD64, Intel 64, EM64T), ARM9, ARM64. Программа, собранная под один ISA, не запустится на другом без перекомпиляции или эмуляции.
3. **Микроархитектура семейства:** размеры кэшей, длина конвейера, число ядер (например, Zen 4 у AMD, Nehalem у Intel).

Пример: Intel Xeon и AMD EPYC — разные микроархитектуры, но один ISA (x86-64), поэтому одна и та же программа работает на обоих.

От кода до сигналов: макро- и микрокоманды

1. **Код пользователя** (Python, C, ассемблер). Компилятор превращает его в машинный код.
2. **Макрокоманды** — инструкции из ISA: `MOV`, `ADD`, `JMP`. Это то, что видит программист и документация. Одна макрокоманда может означать сложное действие: «взять значение из памяти, сложить с регистром, записать обратно».
3. **Микрокоманды (micro-ops, μops)** — процессор декодирует макрокоманду в элементарные действия для исполнительных блоков (АЛУ, блок загрузки/записи, регистры). Именно они реально исполняются в конвейере.

Именно так современные x86 совмещают подходы: снаружи сложные CISC-команды, внутри простые RISC-подобные микрооперации.

CISC, RISC, MISC, VLIW

Подход	Расшифровка	Идея	Примеры (дополнение)
CISC	Complex Instruction Set Computer — полный набор команд	Много сложных команд разной длины; одна команда делает много	x86 (Intel, AMD)
RISC	Reduced Instruction Set Computer — сокращённый набор	Мало простых команд одинаковой длины; легко строить конвейер, меньше энергии	ARM, Apple M, RISC-V
MISC	Minimal Instruction Set Computer — минимальный набор	Ещё меньше команд, часто стековая машина; нишевые и встраиваемые системы	редко встречается
VLIW	Very Long Instruction Word — очень длинная команда	Одна команда содержит несколько операций для параллельного исполнения; параллелизм планирует компилятор, а не железо	Эльбрус, Intel Itanium

Лектор подчёркивает: в чистом виде эти подходы уже не встречаются — в современных системах смешаны элементы разных архитектур.

Топология: сокеты, ядра, потоки

```
\text{логические процессоры} = \text{сокеты} \times \text{физические ядра на сокет} \times \text{потоки на ядро}
```

Пример: 2 процессора × 16 ядер × 2 потока (Hyper-Threading) = 64 логических процессора. ОС видит именно их.

Почему это проблема для ОС. Два потока одного ядра делят его исполнительные блоки, а два ядра одного сокета делят кэш L3. Планировщику нужно знать топологию, чтобы не посадить две тяжёлые задачи на одно физическое ядро. Для этого у каждого логического процессора есть свой **APIC** (Advanced Programmable Interrupt Controller) со своим APIC ID — он отвечает за прерывания, приходящие к этому процессору.

В Linux топологию смотрят в `/proc/cpuinfo`:

Поле	Значение
`processor`	ID логического процессора (с нуля)
`physical id`	ID физического процессора (сокета)
`siblings`	Сколько логических процессоров на этом физическом
`core id`	ID ядра внутри физического процессора
`cpu cores`	Сколько физических ядер в процессоре
`apicid`	ID в Local APIC
`initial apicid`	Исходный APIC ID до изменений BIOS/ОС

Если `siblings` = 2 × `cpu cores`, значит включён Hyper-Threading. Утилита `taskset` привязывает процесс к конкретным логическим процессорам, например `taskset -c 0,1 python train.py`.

Сокет — разъём на плате под процессор. На слайде сравниваются Intel LGA 1151 и AMD Socket AM4. Дополнение: в LGA (Land Grid Array) ножки находятся в сокете, а на процессоре — плоские контакты; AM4 — тип PGA, ножки на самом процессоре. Процессор и плата должны иметь одинаковый сокет.

Кэш-память

Кэш нужен потому, что ОЗУ в десятки раз медленнее процессора. Часто нужные данные держат ближе к ядру: чем ближе уровень, тем он быстрее и меньше.

Уровень	Кому принадлежит	Пример размера
L1	Одному ядру; делится на кэш данных (L1D) и команд (L1I)	Zen 4: 32 КБ D + 32 КБ I
L2	Обычно одному ядру; у Intel E-core — общий на кластер из нескольких ядер	Zen 4: 1 МБ на ядро; E-core: 2–4 МБ на кластер
L3	Общий для всех ядер; самый медленный из кэшей, но быстрее ОЗУ	AMD EPYC (Zen 4): до 1152 МБ

Частота, TDP, конвейер

- **Частота** = базовая частота шины × множитель. Свободный (разблокированный) множитель позволяет менять частоту средствами платы — это нужно для разгона.
- **TDP (Thermal Design Power)** — максимальное тепло, которое должна рассеивать система охлаждения при любой нагрузке. По TDP подбирают кулер и считают питание стойки.
- **Конвейер команд** — несколько инструкций одновременно находятся на разных стадиях: выборка → декодирование → выполнение → доступ к памяти → запись результата. Как конвейер на заводе: пока одна машина на покраске, следующая уже на сварке.
- **Предсказание ветвлений.** На условном переходе (`if`) конвейер не знает, какую команду брать дальше. Процессор угадывает ветку и исполняет её заранее (спекулятивное выполнение). Если ошибся — конвейер сбрасывается и теряются такты.

Реальные процессоры и как их сравнивать

На слайдах показаны AMD EPYC, Intel Xeon и Apple M. Экзотика — **Cerebras Wafer Scale Engine 3**: процессор размером с целую кремниевую пластину, 900 000 ядер, 44 ГБ SRAM, 4 триллиона транзисторов, 23 кВт; по мощности ≈ тысяче NVIDIA V100.

Процессоры сравнивают по четырём критериям: класс (десктопный / серверный), формальные показатели (ядра, частота, кэш, TDP), бенчмарки (например, versus.com) и экономика. **TCO** (Total Cost of Ownership) — полная стоимость владения: покупка + электричество + охлаждение + обслуживание. **ROI** (Return on Investment) — сколько вложение приносит.

GPU: другой подход к вычислениям

CPU — несколько мощных ядер с большим кэшем и сложным управлением, хорошо исполняет разнородный код с ветвлениями. GPU — тысячи простых ядер, которые одновременно делают одно и то же над разными данными. Это идеально для рендеринга, симуляций и машинного обучения, где нужно перемножить огромные матрицы.

Архитектура GPU NVIDIA называется **CUDA** (Compute Unified Device Architecture). Состав GPU:

- **Потоковые процессоры (ядра)** — работают параллельно, есть специализация (например, тензорные ядра для матричных операций).
- **Видеопамять (VRAM)** с несколькими уровнями кэша (L1, L2).
- **Широкая шина данных** между ядрами и VRAM — чтобы тысячи ядер не простаивали.

Иерархия исполнения в CUDA: **сетка (grid)** → **блоки потоков (thread blocks)** → **потоки на ядрах**. У каждого блока есть свой L1 / shared memory, общий для его потоков; L2 общий для всего GPU. На схеме со слайда у CPU большие блоки Control и Cache и 4 ALU, а у GPU — сотни мелких ALU с минимальным управлением.

Материнская плата и чипсет

Процессор сам по себе ничего не может — плата даёт ему питание и доступ к периферии. Характеристики платы: поддерживаемый сокет, питание и тактовые генераторы, поддержка памяти, интерфейсы и шины, микропрограммы (BIOS/UEFI).

Пример со слайда — MSI Z490-A PRO: сокет LGA 1200, чипсет Intel Z490, DDR4 без ECC до 128 ГБ (2133–4800 МГц), 6 × SATA 6 Гбит/с, M.2 NVMe, 2 × PCIe x16, 3 × PCIe x1, звук Realtek ALC892, сеть Realtek RTL8125.

Чипсет обслуживает контроллеры хранения, USB, дополнительные линии PCI Express, Wi-Fi, звук и т. д.

Оперативная память

Характеристики модуля: класс (десктопная UDIMM / серверная RDIMM), тип (DDR3, DDR4, DDR5), объём, частота, тайминги, форм-фактор (DIMM — полноразмерный, SO-DIMM — для ноутбуков), многоканальность.

Дополнение про классы: в **RDIMM** (Registered) есть буферный регистр между контроллером и чипами, поэтому на один канал можно поставить больше памяти. **ECC** (Error-Correcting Code) исправляет одиночные ошибки битов — в серверах это стандарт.

Тайминги — задержки в тактах между командой и её исполнением, обычно 4 числа (например, 16-18-18-38). Память устроена как таблица: чтобы прочитать ячейку, нужно открыть строку (row, RAS) и выбрать столбец (column, CAS).

Тайминг	Что означает
CAS Latency (CL) — главный	Такты от запроса до начала ответа
RAS to CAS Delay (tRCD)	Такты на активацию нужной строки банка
RAS Precharge (tRP)	Такты на закрытие строки и переход к другой
Row Active Time (tRAS)	Такты, пока строка открыта, до её закрытия

Важно: тайминги считаются в тактах, а не в наносекундах. У быстрой памяти такт короче, поэтому CL 40 на DDR5 может быть не медленнее CL 16 на DDR4. Итоговая скорость = f(частота, тайминги, число каналов), и память, плата и CPU должны друг другу соответствовать.

Характеристика	DDR3	DDR4	DDR5
Частота, МГц	800–1600	2133–3200+	4800–8400+
Пропускная способность	~12–15 ГБ/с	~17–25 ГБ/с	~38–67+ ГБ/с
Напряжение, В	1,5 / 1,35	1,2	1,1
Макс. объём модуля	до 8 ГБ	до 16–32 ГБ	до 64–128 ГБ
Каналы в модуле	один 64-битный	один 64-битный	два 32-битных
Предварительная выборка	8n	8n	16n

Двухканальный режим: если поставить модули в слоты разных каналов, контроллер читает из них параллельно, и пропускная способность удваивается. Два модуля по 8 ГБ быстрее одного на 16 ГБ.

Форм-факторы серверов

- **Tower** — отдельно стоящий корпус, как у домашнего ПК. Для маленьких офисов.
- **Rack-mount** — сервер стандартной ширины (19 дюймов) для стойки. Дополнение: высота измеряется в юнитах, 1U = 44,45 мм; типичная стойка — 42U.

Выводы лекции 2

- В современных системах смешаны элементы разных архитектур (фон Нейман + Гарвард, CISC снаружи + RISC внутри).
- «Архитектура CPU» имеет три разных смысла: подход, набор команд, микроархитектура.
- У GPU своя ниша — массово параллельные вычисления.
- Основа модульной архитектуры — материнская плата.
- DDR5 лучше.

Лекция 5, часть 1. Слой Storage: диски, интерфейсы, протоколы

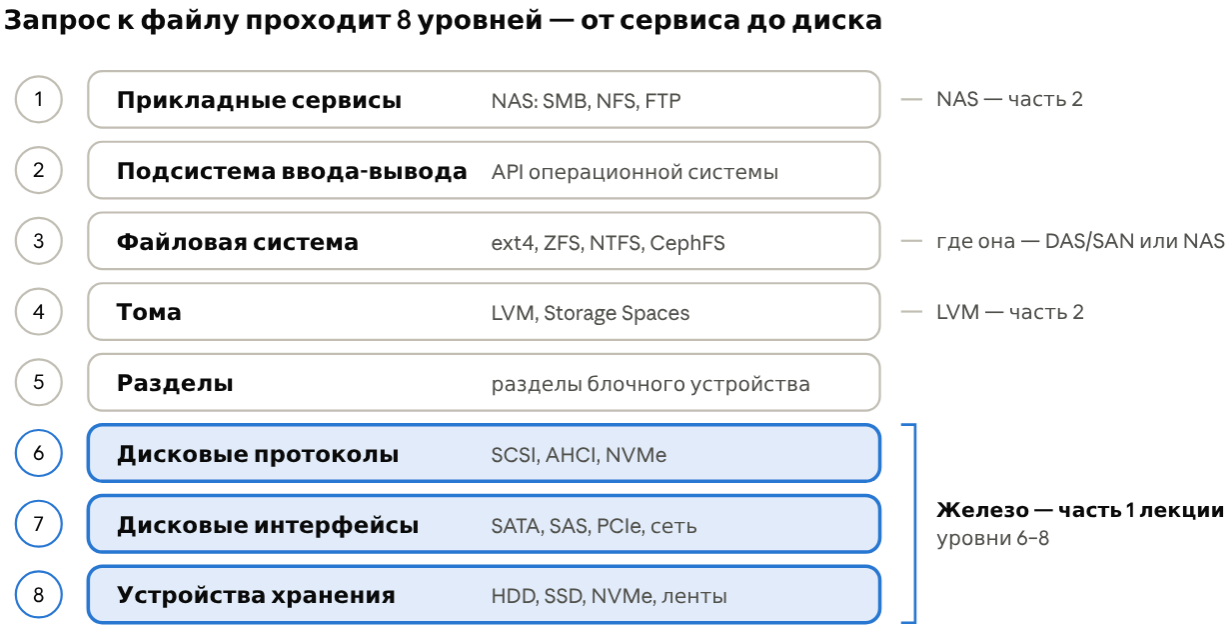
Хранение данных устроено как стек из 8 уровней. Эта часть разбирает нижние три — железо: устройства, интерфейсы и дисковые протоколы. Главный сюжет: HDD → SSD, который «прикидывается» HDD → NVMe, который перестал прикидываться.

Задачи слоя Storage

Storage — инфраструктура хранения: устройства, DAS/NAS/SAN, абстрагированные и облачные сервисы. Кроме самого хранения он обеспечивает:

- надёжность — данные не теряются при отказе диска;
- производительность под требования приложений;
- версионирование (снапшоты, история изменений);
- масштабирование;
- безопасность;
- экономическую эффективность.

Обобщённая архитектура стека хранения



обобщённая архитектура стека хранения · 8 уровней

Как понять: когда Python-скрипт вызывает `open('data.csv').read()`, запрос проходит вниз через все уровни. ОС (2) отдаёт его файловой системе (3), та превращает имя файла в номера блоков на томе (4), том — в блоки раздела (5), дальше команда по протоколу (6) идёт по кабелю (7) к диску (8). Это как стек TCP/IP, только для дисков: протоколы — аналог канального уровня, отвечают за адресацию, приоритизацию и коммутацию.

HDD: жёсткий магнитный диск

Устройство: пластины (диски), двигатель-шпиндель, головки чтения/записи (по одной на поверхность), энергозависимый кэш, контроллер, питание, интерфейс.

Адресация. Данные лежат на дорожках (концентрических кольцах), дорожки делятся на сектора.

- **Сектор** — минимальный адресуемый блок диска: 512 байт на старых HDD, 4096 байт (4 КБ) на дисках Advanced Format.
- **Кластер** — абстракция файловой системы, не диска. Кратен сектору: при секторе 4 КБ кластер не меньше 4 КБ.

Контроллер компьютера видит диск как устройство с прямым доступом — длинный массив пронумерованных блоков. ОС просит «записать в кластер N», а контроллер сам превращает это в физические операции над секторами.

Характеристики HDD:

Характеристика	Значения	Что даёт
Скорость вращения	4200, 5400, 7200, 10 000, 15 000 об/мин	Быстрее вращение — меньше ждать нужный сектор
Среднее время ожидания	2–14 мс	Главная слабость HDD на случайном доступе
Буфер (кэш)	8–256 МБ	Помогает на маленьких файлах
MTBF (наработка на отказ)	у серверных ~1 млн часов	Важно для серверов
Ударостойкость	в единицах перегрузки G, отдельно вкл. и выкл.	Важно для ноутбуков
Также	интерфейс, форм-фактор (3,5″ / 2,5″), шум	

Почему HDD медленный на случайном доступе: каждый раз головка должна физически переехать на нужную дорожку и дождаться, пока под ней прокрутится сектор. Именно это и случилось в истории с ML-стартапом из лекции 1: миллионы мелких фотографий = миллионы перемещений головки.

Объёмы со слайдов: Seagate ST14000DM001 — 14 ТБ, Seagate Exos M — 36 ТБ.

SSD: твердотельный накопитель

Устройство: энергонезависимая флеш-память NAND, энергозависимый кэш (бывают без него), контроллер, питание, интерфейс. Движущихся частей нет, поэтому случайный доступ почти такой же быстрый, как последовательный.

Адресация и «заморочное» удаление:

- NAND физически организована в **блоки** (обычно 128 КБ и больше), блоки делятся на **страницы** (обычно 4 КБ).
- Записывать можно постранично, но **только в чистую страницу**: перезаписать поверх нельзя.
- Стирать можно **только целый блок**.
- Файловая система по-прежнему работает с кластерами, как с HDD, а контроллер SSD внутри переводит это в операции с ячейками. В этом смысле SSD «прикидывается» HDD.

Как понять: блок — как тетрадь, написанная ручкой. Исправить одну страницу нельзя, можно лишь переписать её в другую тетрадь и пометить старую как недействительную. Когда в тетради много зачёркнутых страниц, живые переносят в новую, а старую стирают целиком.

Именно этим занимается контроллер:

- **Garbage Collection (сборка мусора)** — фоново собирает живые страницы из полупустых блоков и освобождает блоки.
- **TRIM** — команда от ОС: «вот эти блоки больше не нужны, файл удалён». Без TRIM контроллер не знает об удалении и зря таскает мёртвые данные.
- Побочный эффект (дополнение): **усиление записи (write amplification)** — на каждый записанный пользователем байт диск физически пишет больше из-за переноса живых страниц.

Характеристики SSD: интерфейс (SATA/SAS или NVMe), тип NAND, форм-фактор (2,5″, толщина, M.2), объём, наличие и объём буфера, MTBF, операций в секунду (IOPS), ударостойкость, энергопотребление и тепловыделение, **число циклов перезаписи**.

Типы NAND и ресурс SSD

Ячейка NAND хранит заряд. Чем больше бит в одной ячейке, тем больше уровней заряда нужно различать, и тем быстрее ячейка изнашивается. Зато дешевле гигабайт.

Тип	Бит на ячейку	Ресурс, циклов P/E	Стоимость за ГБ
SLC	1	~50 000	самая высокая
MLC	2	~3 000	высокая
TLC	3	~3 000	ниже
QLC	4	~1 000	самая низкая

P/E (Program/Erase) — один цикл «записали — стёрли». Пример потребительского QLC: Samsung 870 QVO на 8 ТБ.

Расчёт ресурса со слайда. Диск 120 ГБ, в день пишется 12 ГБ, усиление записи — 10×. Значит физически в день пишется 120 ГБ, то есть ровно 1 полный цикл перезаписи.

```
\text{срок службы (дней)} = \frac{\text{ресурс циклов}}{\text{циклов в день}}
= \frac{\text{ресурс циклов} \times \text{ёмкость}}{\text{запись в день}
\times \text{усиление записи}}
```

При 3 000 циклах: 3 000 дней ≈ 8 лет. При 5 000 циклах: ≈ 13,5 лет. Вывод: при обычной нагрузке SSD живёт дольше, чем нужен; риск — для серверов с постоянной записью (логи, БД) на QLC-дисках.

NVMe-накопители и их форм-факторы

NVMe-SSD подключается напрямую к шине PCIe. Форм-факторы:

- **M.2** — маленькая планка длиной около 8 см, как в ноутбуках и домашних ПК;
- **E1.S и E1.L** (EDSFF, «линейки») — серверные форматы; E1.L длиной около 32 см. Позволяют плотно набить сервер дисками и хорошо охлаждать их.

Зачем всё ещё нужны HDD

Из-за цены за терабайт. Мем со слайда: 1 ТБ SSD — быстрый Флэш, 8 ТБ HDD за те же деньги — медленный, но большой. На практике делают **tiering (ярусное хранение)**: «горячие» данные на SSD/NVMe, «холодные» на HDD, архивы на ленте.

Экзотика: когда данные проще отвезти

Устройство	Ёмкость	Особенности
WD Ultrastar Transporter	368 ТБ	«Флешка в чемодане» на колёсиках, 15 кг. Внутри целый компьютер: Xeon 4310 на 12 ядер, 128 ГБ RDIMM, сеть 200GbE и 10GbE, FIPS 140-2, TPM 2.0
AWS Snowmobile	100 ПБ	Грузовик-хранилище: 10 машин перевозят экзабайт примерно за полгода (по каналу 10 Гбит/с — около 26 лет). 350 кВт, шифрование, GPS, вооружённая охрана
HPE StoreEver LTO Ultrium	15 ТБ со сжатием	Ленточный накопитель (стример): SAS 6 Гбит/с, 300 МБ/с, AES-256, срок хранения 30 лет

Лента жива потому, что для архивов важны цена и срок хранения, а не скорость: доступ строго последовательный.

Дисковые интерфейсы: SATA, SAS, NVMe

Кто через что подключается: HDD — PATA и SCSI (устарели), SATA, SAS; SSD — SATA, SAS, NVMe.

	SATA	SAS
Расшифровка	Serial ATA	Serial Attached SCSI
Предок	Параллельный ATA (IDE)	Параллельный SCSI
Версии и скорости	1.0 — 1,5 Гбит/с (2003); 2.0 — 3 Гбит/с (2005); 3.0 — 6 Гбит/с (2008)	SAS-1 — 3 Гбит/с (2004); SAS-2 — 6 (2009); SAS-3 — 12 (2013); SAS-4 — 22,5 «24G» (2017)
Режим передачи	Полудуплекс: либо чтение, либо запись	Полный дуплекс: в обе стороны одновременно
Адресация	По физическим портам	У каждого устройства свой адрес
Число устройств	Одно на порт	Более 16 384 (у старого SCSI — 8, 16 или 32 на шину)
Протокол	AHCI	SCSI
Hot Plug	да	да
Совместимость	—	SATA-диск можно включить в SAS-контроллер, наоборот нельзя

Hot Plug — диск можно вынуть и вставить на работающем сервере. По фото разъёмов на слайде: у SATA в разъёме прорезь, у SAS на её месте перемычка с контактами. Поэтому SAS-диск физически не вставится в SATA-коннектор.

Компоненты SAS (эти же термины встретятся в SAN):

- **Инициатор (initiator)** — порождает запросы к целевым устройствам и получает подтверждения. Это контроллер (HBA) или программная сущность.
- **Целевое устройство (target)** — содержит логические блоки, принимает и исполняет запросы, отсылает подтверждение. Отдельный диск или целый дисковый массив.
- **Подсистема доставки данных (Service Delivery Subsystem)** — кабели между инициатором и таргетом, иногда с экспандерами.

- **Экспандер (expander)** — позволяет подключить несколько таргетов к одному порту инициатора, полностью прозрачно для них. Аналогия — сетевой коммутатор.

Типичная схема со слайда: сервер → HBA-контроллер → кабели → экспандер → дисковая полка.

NVMe (Non-Volatile Memory Express) — интерфейс и протокол, созданный специально для SSD на шине PCIe.

Версия NVMe	Шина	Чтение, до	Запись, до
1.0 / 1.3	PCIe 3.0 x4	3,5 ГБ/с	3 ГБ/с
1.4	PCIe 4.0 x4	7 ГБ/с	5,5 ГБ/с
2.0	PCIe 5.0 x4	14 ГБ/с	10 ГБ/с

Для сравнения: SATA 3.0 — 6 Гбит/с, то есть примерно 0,6 ГБ/с. Обратите внимание на единицы: интерфейсы обычно указывают в гигабитах (Гбит/с), диски — в гигабайтах (ГБ/с); 1 байт = 8 бит.

Дисковые протоколы: AHCI, SCSI, NVMe

Интерфейс — это «дорога» (разъёмы, кабели, сигналы), протокол — «правила дорожного движения»: какие команды и в каком формате по ней едут. Пары: SATA → AHCI, SAS → SCSI, PCIe → NVMe.

	AHCI	SCSI	NVMe
Создан для	HDD	HDD	флеш-памяти
Очереди команд	1 очередь × 32 команды	очереди длиной 256, их может быть несколько	до 65 536 очередей × 65 536 команд
Накладные расходы	высокие	высокие	низкие: меньше инструкций CPU, низкие задержки PCIe
Привязка к интерфейсу	жёстко привязан к SATA	не привязан к SAS	не привязан к PCIe
Работа по сети	нет	iSCSI (поверх TCP/IP)	NVMe over TCP

Почему очереди так важны. У HDD одна головка, он всё равно исполняет команды по одной, и очереди в 32 команды ему хватает. SSD состоит из десятков чипов NAND, которые работают параллельно, а серверный CPU имеет десятки ядер. NVMe даёт каждому ядру свою очередь, и все чипы загружены одновременно. SSD через AHCI — как многополосная трасса с одной кассой на въезде.

Инкапсуляция. SCSI и NVMe можно упаковать в сетевые пакеты TCP/IP: так получаются iSCSI и NVMe over TCP. Диск на другом конце сети выглядит для сервера как локальный. Это основа SAN из части 2.

Итоги части 1

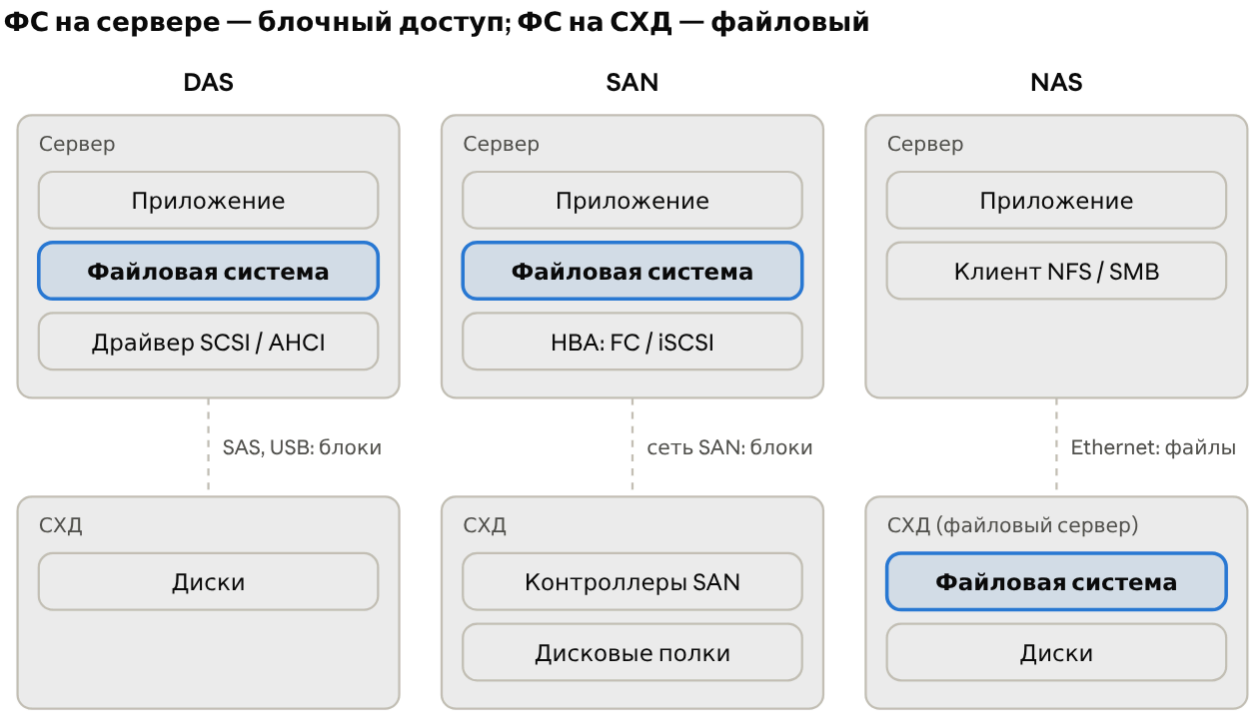
- Старые устройства хранения — HDD. SSD «прикидывается» HDD (те же интерфейсы и протоколы). NVMe перестал прикидываться.
- Интерфейсы (SATA, SAS, PCIe) — «канальный» уровень хранения, и он может быть сложным (экспандеры, полки).
- Протоколы (AHCI, SCSI, NVMe) работают поверх интерфейсов, но SCSI и NVMe не привязаны к конкретному интерфейсу и умеют ходить по сети.

Лекция 5, часть 2. Слой Storage: DAS / NAS / SAN, RAID, LVM, измерение скорости

В корпоративном мире хранилище чаще всего вынесено из сервера в отдельную систему хранения данных (СХД). Как именно вынесено — DAS, NAS или SAN — определяет, на каком уровне стека хранения сервер и СХД «встречаются». Главная задача слоя — избыточность, традиционно через RAID, а всё чаще на уровне томов.

Четыре способа подключить хранилище

- **DAS** (Direct Attached Storage) — напрямую к серверу.
- **NAS** (Network Attached Storage) — файловый сервер в сети.
- **SAN** (Storage Area Network) — отдельная сеть для блочного доступа.
- **SDS** (Software Defined Storage) — хранилище, собранное программно из обычных серверов с дисками (например, Ceph). На лекции только упомянуто.



DAS, SAN и NAS · где проходит граница между сервером и СХД

Ключ к пониманию: **где живёт файловая система**. В DAS и SAN она на сервере, а СХД отдаёт «сырые» блоки (block I/O) — сервер видит её как локальный диск. В NAS файловая система на самой СХД, и сервер просит уже файлы (file I/O) — как сетевую папку.

DAS — прямое подключение

СХД только хранит данные и даёт к ним доступ, подключена непосредственно к вычислительному узлу; всё остальное (файловая система, иногда RAID) делается на стороне узла. Пример: дисковая полка DEPO Storage 2312 (DAS SAS3); два сервера с SAS HBA через экспандеры подключены к одной JBOD-полке Supermicro.

Интерфейс DAS	Скорость
USB	до 10–20 Гбит/с
eSATA	как SATA 2 (3 Гбит/с)
SAS	по версии SAS
FireWire (IEEE 1394)	400–3200 Мбит/с
Thunderbolt	до 20 Гбит/с

Плюсы DAS	Минусы DAS
Низкая стоимость	Низкая надёжность: авария сервера — данные недоступны всем сразу
Простота развёртывания и администрирования	Высокая латентность для других: все запросы идут через один сервер
Высокая скорость обмена между массивом и сервером	Высокая загрузка сети, если сервер раздаёт данные другим
	Плохая управляемость и утилизация: ёмкость привязана к одному серверу; у одних избыток, у других нехватка
	Нельзя совместно использовать СХД несколькими серверами

NAS — файловый сервер в сети

СХД — полноценный файловый сервер. Подключение через обычную сеть, доступ — через сетевые файловые протоколы: **NFS** (привычен для Linux), **SMB** (привычен для Windows), FTP. Высокая степень абстракции: клиент работает с верхним уровнем стека (уровень 1 — прикладные сервисы).

Плюсы NAS	Минусы NAS
Доступность и низкая стоимость	Данные хранятся только как файлы
Простое совместное использование	Медленнее локального диска из-за сетевых протоколов
Простота подключения и управления	Высокая загрузка локальной сети
Гибкость: объём быстро расширяется	Некоторые приложения не работают с сетевыми дисками (например, СУБД часто требуют блочное устройство)
Универсальность: клиент на любой ОС	

ОС для NAS:

- общего назначения: Linux, FreeBSD, Windows Storage Server;
- специализированные enterprise: Open-E DSS, RAIDIX;

- для дома и малого офиса (SOHO): FreeNAS (на FreeBSD, самая распространённая), OpenMediaVault (на Debian), XPEnology (сборка Synology DSM), Rockstor (на CentOS).

NFS vs SMB. На графике со слайда NFS, SMB и SSHFS показывают почти одинаковую скорость последовательного чтения и записи — около 110 МБ/с. Причина (дополнение): тест шёл через гигабитную сеть, а 1 Гбит/с ≈ 125 МБ/с. Узким местом был не протокол, а сеть. Поэтому серьёзные NAS ставят на 10/40 Гбит/с (например, DEPO Storage 3516: до 96 ТБ, сеть 1/10/40 Гбит/с).

SAN — сеть хранения данных

SAN — комплексное решение поверх специализированной инфраструктуры. СХД состоит из дисковых полок и контроллеров SAN. Подключение через сеть, но доступ **не к файлам, а к блочным устройствам**: SCSI-команды инкапсулируются в сетевые сообщения.

Как понять: SAN — это DAS, в котором кабель от сервера к дискам заменили на сеть. Сервер по-прежнему видит «свой» диск, но этот диск можно передать другому серверу, а ёмкость общая на всех.

Плюсы SAN	Минусы SAN
Масштабирование	Нужна специальная инфраструктура
Производительность	Специализированные SAN стоят очень дорого
Сложные задачи хранения «из коробки» (снапшоты, репликация и т. п.)	

Транспорт SAN:

- по сетям общего назначения (Ethernet): **iSCSI** — SCSI поверх TCP/IP;
- по специализированным канальным протоколам: **Fibre Channel (FC)**, **InfiniBand (IB)**.

Компоненты SAN: узлы (nodes) — дисковые массивы (таргеты) и серверы-потребители (инициаторы); сетевая инфраструктура — коммутаторы, маршрутизаторы, кабели. **Фабрика (fabric)** — коммутаторы, соединённые межкоммутаторными линками (ISL, InterSwitch Link). Примеры железа: коммутатор HP StorageWorks 4/8 SAN Switch, FC HBA HP 82E 8Gb, СХД Dell PowerVault ME4024, российская YADRO TATLIN.

Fibre Channel по поколениям:

Интерфейс	Пропускная способность	Линейная скорость	Кодирование	Хост-адаптер
1Gb FC	100 МБ/с	1,0625 ГБод	8b/10b	PCI-X
2Gb FC	200 МБ/с	2,125 ГБод	8b/10b	PCI-X
4Gb FC	400 МБ/с	4,25 ГБод	8b/10b	PCI-X 2.0 / PCIe 1.0 x4
8Gb FC	800 МБ/с	8,5 ГБод	8b/10b	PCI-X 1.0 x8 / PCIe 2.0 x4
16Gb FC	1600 МБ/с	14,025 ГБод	64b/66b	PCI-X 2.0 x8 / PCIe 3.0 x4
32Gb FC	3200 МБ/с	28,05 ГБод	64b/66b	PCIe 3.0 x8
64Gb FC	6400 МБ/с	28,9 ГБод	64b/66b	PCIe 4.0

Как читать кодирование (дополнение): 8b/10b означает, что 8 бит данных передаются 10 битами на линии — 20% уходит на служебную информацию. Поэтому у 1Gb FC линейная скорость 1,0625 ГБод, а полезная — ровно 100 МБ/с. Переход на 64b/66b снизил потери до ≈ 3%.

Fibre Channel vs iSCSI (по схеме со слайда). Над всеми вариантами один и тот же слой SCSI, различается только транспорт внизу:

Вариант	Стек под SCSI	Физическая среда
FC	напрямую FC	выделенная сеть FC (1–64 Гбит/с)
FCIP	FC → TCP → IP	Ethernet (связать две FC-сети через интернет)
FCoE	FC напрямую в Ethernet-кадрах	усовершенствованный Ethernet (CEE)
iSCSI	TCP → IP	обычный Ethernet
SRP / iSER	RDMA	InfiniBand

iSCSI дешевле (обычные сетевые карты и коммутаторы), FC — предсказуемее и быстрее, но требует отдельной инфраструктуры. У iSCSI есть вариант с TOE (TCP Offload Engine) — сетевая карта сама обрабатывает TCP, разгружая CPU.

Сводное сравнение DAS, NAS, SAN

	DAS	NAS	SAN
Что отдаёт СХД	блоки	файлы	блоки
Где файловая система	на сервере	на СХД	на сервере
Подключение	кабель напрямую (SAS, USB…)	обычная сеть (Ethernet)	выделенная сеть (FC, iSCSI, IB)
Протоколы	SCSI, AHCI	NFS, SMB, FTP	iSCSI, FC
Совместный доступ серверов	нет	да, к файлам	да, серверам выдаются тома
Стоимость	низкая	низкая-средняя	высокая
Типичное применение	один сервер, много дисков	общие папки, документы, датасеты	БД, виртуализация, критичные системы

Избыточность

Самое ценное в современных ИС — данные. **Избыточность** — выполнение одинаковых функций несколькими компонентами, в первую очередь ради надёжности. **Надёжность** — способность системы сохранять функциональность целиком или частично.

Избыточность обеспечивается на уровне дисков (RAID), тома (LVM), файловой системы (например, ZFS), данных, Middleware (репликация СУБД) и приложений.

RAID

RAID (Redundant Array of Independent Disks) — базовый способ избыточности: несколько дисков объединяются в один логический.

- **Аппаратный RAID** — создаётся и обслуживается контроллером без участия ОС. ОС видит один диск и может использоваться только для диагностики. Настраивается в BIOS контроллера (на слайдах — утилиты Adaptec и LSI, вход по Ctrl+G).
- **Программный RAID** — создаётся и обслуживается ОС (в Linux — mdadm).

Уровни RAID предложены в Калифорнийском университете в Беркли и стали стандартом де-факто. В таблице N — число дисков; столбцы с ёмкостью и минимумом дисков — дополнение.

Уровень	Как устроен	Мин. дисков	Полезная ёмкость	Переживает отказ
JBOD	Диски просто склеены последовательно, не RAID	1	сумма дисков	0 дисков
RAID 0	Чередование (striping): блоки по очереди на все диски. Максимум скорости, без защиты	2	N	0
RAID 1	Зеркало: одинаковые данные на двух дисках	2	N / 2	1 из пары
RAID 2	Код Хэмминга, на практике не используется	—	—	—
RAID 3, 4	Чередование + выделенный диск чётности	3	N − 1	1
RAID 5	Чередование + чётность, размазанная по всем дискам	3	N − 1	1
RAID 6	Чередование + две независимые чётности	4	N − 2	2
RAID 10	RAID 0 из зеркал RAID 1	4	N / 2	по 1 в каждом зеркале
RAID 50	RAID 0 из массивов RAID 5	6	N − число групп	по 1 в каждой группе
RAID 60	RAID 0 из массивов RAID 6	8	N − 2 × число групп	по 2 в каждой группе

JBOD vs RAID 0. В JBOD файл лежит на одном диске (A1–A6 на Disk 0), диски просто заполняются по очереди. В RAID 0 блоки A1, A2, A3, A4 разнесены по четырём дискам и читаются одновременно — отсюда скорость. Расплата: отказ любого диска убивает весь массив.

Как RAID 5 восстанавливает данные. Для каждой полосы (stripe) считается блок чётности через XOR (исключающее ИЛИ):

```
X_p = X_1 \oplus X_2 \oplus X_3
```

У XOR есть свойство: если известны все члены, кроме одного, потерянный равен XOR остальных. В примере со слайда сломан Disk 1: A2 = XOR(A1, A3, Ap), B2 = XOR(B1, Bp, B3), D1 = XOR(Dp, D2, D3).

Разбор на битах (дополнение): A1 = 1010, A2 = 0110, A3 = 1100. Тогда Ap = 1010 ⊕ 0110 ⊕ 1100 = 0000. Потеряли A2 — считаем 1010 ⊕ 1100 ⊕ 0000 = 0110. Данные вернулись.

Чётность в RAID 5 «гуляет» по дискам (Ap на Disk 3, Bp на Disk 2, Cp на Disk 1…), чтобы ни один диск не стал узким местом при записи. Именно в этом его отличие от RAID 3/4 с выделенным диском чётности.

RAID 10 vs RAID 5 на 4 дисках (дополнение): RAID 5 даёт 3 диска полезного объёма, RAID 10 — только 2. Зато RAID 10 быстрее на запись (не нужно считать чётность) и быстрее восстанавливается (просто копируется зеркало). Поэтому под БД чаще берут RAID 10, под файловые архивы — RAID 5/6.

Тренд: аппаратные RAID всё ещё широко используются, но постепенно уступают место избыточности на уровне томов. В SDS их иногда прямо запрещают: SDS сам распределяет копии данных и должен видеть каждый диск напрямую.

Управление томами: LVM

Том — система управления дисковым пространством, отвязанная от физических устройств. У тома легко менять размер, его можно сделать избыточным и снять снимок состояния; он масштабируется, упрощает систему и не зависит от железа. В Linux этот слой представлен LVM (Logical Volume Manager), в Windows — Storage Spaces.

Понятие	Что это
PV, Physical Volume	Физический том: раздел диска или весь диск, в том числе программный или аппаратный RAID
VG, Volume Group	Группа томов: состоит из PV с одной стороны и из LV с другой. Единая административная единица, «общий котёл» места
LV, Logical Volume	Логический том: кусок VG, аналог раздела диска; на нём создаётся файловая система
PE, Physical Extent	Порция, на которые нарезан каждый PV

Понятие	Что это
LE, Logical Extent	Порция, на которые нарезан каждый LV; размер равен PE и одинаков в пределах VG

Иерархия снизу вверх: диски `/dev/sda`, `/dev/sdb`, `/dev/sdc` → PV → одна VG `/dev/my-lvm-vg` → LV `/` и `/home`. Каждый LE отображается на какой-то PE. Как понять: диски сливают в общий котёл (VG), а из него нарезают логические диски нужного размера. Не хватает места в `/home` — докидываешь диск в VG и расширяешь LV, без переразметки.

Способы отображения экстентов:

- **Линейное** — LE 1–4 идут на первый PV (заполнен на 100%), LE 5–8 на второй (заполнен на 50%). Аналог JBOD.
- **Чередующееся (striped)** — нечётные LE на первый PV, чётные на второй; оба заполнены на 75%. Аналог RAID 0: быстрее, но отказ одного PV убивает весь LV.

Снапшоты — мгновенные снимки состояния тома. Команда со слайда (там опечатка `lcreate`):

```
lvcreate -L 1G -s -n snapshot_vg_data /dev/mapper/vg_lv_data
# -L 1G  — размер области под изменения; -s — snapshot; -n — имя
```

Типичные команды LVM (дополнение, пригодятся на практиках):

```
pvcreate /dev/sdb /dev/sdc              # диски → физические тома
vgcreate my-vg /dev/sdb /dev/sdc        # PV → группа томов
lvcreate -L 20G -n data my-vg           # линейный LV на 20 ГБ
lvcreate -L 20G -i 2 -n fast my-vg      # чередующийся LV на 2 PV
lvextend -r -L +10G /dev/my-vg/data     # увеличить LV вместе с ФС
pvs; vgs; lvs                           # посмотреть состояние
```

Как измерять скорость дисковой подсистемы

Скорость зависит не только от диска и интерфейса, но и от характера нагрузки. Измеряют пять показателей: линейное чтение, случайное чтение, линейную запись, случайную запись и IOPS.

- **Линейные (последовательные) операции** — части файла читаются одна за другой; передача больших файлов, блоки больше 128 КБ. Пример: запись видео, бэкап.

- **Случайные (произвольные) операции** — данные читаются из разных мест носителя, обычно блоками по 4 КБ. Пример: БД, миллионы мелких картинок для обучения модели.

IOPS (Input/Output Operations Per Second) — число операций ввода-вывода в секунду, то есть сколько блоков успевает прочитаться или записаться. Один из ключевых параметров для HDD, SSD и SAN. Чтобы связать IOPS со скоростью, нужно знать размер блока:

```
\text{скорость (МБ/с)} = \text{IOPS} \times \text{размер блока}
```

Пример: 100 000 IOPS × 4 КБ ≈ 400 МБ/с. А диск, выдающий 3 ГБ/с на блоках 4 МБ, делает всего ≈ 750 IOPS. Поэтому цифра IOPS без размера блока и типа операции ничего не значит.

Параметр	Описание
Total IOPS	Суммарно операций в секунду (чтение + запись)
Random Read IOPS	Среднее число операций случайного чтения
Random Write IOPS	Среднее число операций случайной записи
Sequential Read IOPS	Среднее число операций линейного чтения
Sequential Write IOPS	Среднее число операций линейной записи

От чего зависит IOPS: конструкция и настройка дисков и RAID; драйвер; драйвер файловой системы; ОС; условия теста — соотношение чтений и записей, размеры блоков, число потоков, размеры очередей и буферов, фрагментация ФС, фоновые приложения.

Что видно на замерах firstVDS (значения приблизительные, сняты с графиков на слайдах):

- На маленьких блоках (4 КБ) много IOPS, но мало МБ/с; на больших (1–4 МБ) наоборот. Это прямо следует из формулы выше.
- NVMe на чтении большими блоками доходит примерно до 2,9 ГБ/с, SATA SSD упирается в ≈ 0,55 ГБ/с — потолок интерфейса SATA 6 Гбит/с.
- На случайном чтении 4 КБ NVMe даёт порядка 150 тыс. IOPS, SATA SSD — порядка 70 тыс.

Дополнение: в Linux стандартный инструмент для таких замеров — `fio`:

```
# случайное чтение блоками 4 КБ, глубина очереди 32
fio --name=randread --rw=randread --bs=4k --iodepth=32 --size=1G --direct=1 -
-runtime=30
# линейное чтение блоками 1 МБ
fio --name=seqread --rw=read --bs=1M --iodepth=8 --size=1G --direct=1 --
runtime=30
```

Выводы части 2

- Важная задача слоя Storage — избыточность. Традиционное решение — RAID.
- Избыточность реализуется и на уровне томов (LVM).
- Хранение в enterprise чаще всего выносное: DAS, NAS или SAN.
- Скорость нужно измерять под свою нагрузку: IOPS и МБ/с связаны через размер блока.

Глоссарий

Все аббревиатуры и термины курса по алфавиту; в столбце «Где» — лекция, в которой термин разобран.

Термин	Расшифровка	Кратко	Где
AHCI	Advanced Host Controller Interface	Дисковый протокол SATA; 1 очередь на 32 команды	Л5.1
APIC	Advanced Programmable Interrupt Controller	Контроллер прерываний логического процессора, у каждого свой APIC ID	Л2
CAS Latency (CL)	Column Address Strobe Latency	Главный тайминг ОЗУ: такты от запроса до начала ответа	Л2
CIA	Confidentiality, Integrity, Availability	Триада безопасности: конфиденциальность, целостность, доступность	Л1
CISC	Complex Instruction Set Computer	Подход с множеством сложных команд (x86)	Л2
CUDA	Compute Unified Device Architecture	Архитектура и платформа вычислений на GPU NVIDIA	Л1, Л2

Термин	Расшифровка	Кратко	Где
DAS	Direct Attached Storage	Хранилище, подключённое напрямую к серверу	Л5.2
DDR	Double Data Rate	Тип ОЗУ (DDR3/4/5), передаёт данные дважды за такт	Л2
DIMM / SO-DIMM	(Small Outline) Dual In-line Memory Module	Форм-факторы модулей памяти: полноразмерный и ноутбучный	Л2
ECC	Error-Correcting Code	Память с исправлением ошибок, стандарт для серверов	Л2
E1.S / E1.L	EDSFF	Серверные форм-факторы NVMe-накопителей («линейки»)	Л5.1
ELK	Elasticsearch, Logstash, Kibana	Стек для сбора, поиска и визуализации логов	Л1
FC	Fibre Channel	Специализированный транспорт для SAN, 1–64 Гбит/с	Л5.2
FCH	Fusion Controller Hub	Чипсет (бывший южный мост) у AMD	Л2
GPU	Graphics Processing Unit	Графический процессор: тысячи простых ядер для параллельных вычислений	Л2
HBA	Host Bus Adapter	Контроллер в сервере для подключения дисков или SAN (SAS HBA, FC HBA)	Л5.1, Л5.2
HDD	Hard Disk Drive	Жёсткий магнитный диск	Л5.1
Hot Plug	—	Замена диска без выключения сервера	Л5.1
IaaS / PaaS / SaaS	Infrastructure / Platform / Software as a Service	Облачные модели: до какого слоя всё делает провайдер	Л1

Термин	Расшифровка	Кратко	Где
IB	InfiniBand	Специализированная высокоскоростная сеть для SAN и кластеров	Л5.2
IOPS	Input/Output Operations Per Second	Операций ввода-вывода в секунду	Л5.2
ISA	Instruction Set Architecture	Набор команд процессора (x86-64, ARM64)	Л2
iSCSI	Internet SCSI	SCSI-команды поверх TCP/IP	Л5.1, Л5.2
ISL	InterSwitch Link	Линк между коммутаторами фабрики SAN	Л5.2
JBOD	Just a Bunch Of Disks	Диски без RAID, склеенные последовательно; также дисковая полка без контроллера	Л5.2
LGA / PGA	Land / Pin Grid Array	Типы сокетов: ножки в сокете или на процессоре	Л2
LVM	Logical Volume Manager	Менеджер томов в Linux	Л5.2
LV / VG / PV	Logical Volume / Volume Group / Physical Volume	Уровни LVM: логический том, группа, физический том	Л5.2
LE / PE	Logical / Physical Extent	Порции, на которые нарезаны LV и PV	Л5.2
LTO	Linear Tape-Open	Стандарт ленточных накопителей (стримеров)	Л5.1
M.2	—	Компактный форм-фактор накопителей (~8 см)	Л2, Л5.1
Middleware	—	Промежуточное ПО между приложением и ресурсами	Л1
MISC	Minimal Instruction Set Computer	Подход с минимальным набором команд	Л2

Термин	Расшифровка	Кратко	Где
MOM	Message-Oriented Middleware	Брокеры сообщений: Kafka, RabbitMQ	Л1
MTBF	Mean Time Between Failures	Средняя наработка на отказ	Л5.1
NAND (SLC/MLC/TLC/QLC)	—	Тип флеш-памяти; 1/2/3/4 бита на ячейку	Л5.1
NAS	Network Attached Storage	Файловое хранилище в сети	Л5.2
NFS / SMB	Network File System / Server Message Block	Сетевые файловые протоколы (Linux / Windows)	Л5.2
NIST	National Institute of Standards and Technology	Автор Cloud Computing Reference Architecture — модели стека курса	Л1
NVMe	Non-Volatile Memory Express	Интерфейс и протокол для SSD на шине PCIe; до 65 536 очередей	Л5.1
P/E-цикл	Program/Erase	Один цикл записи-стирания ячейки NAND	Л5.1
PCH	Platform Controller Hub	Чипсет (бывший южный мост) у Intel	Л2
PCIe	PCI Express	Высокоскоростная шина для GPU, NVMe, сетевых карт	Л2, Л5.1
RAID	Redundant Array of Independent Disks	Массив дисков с избыточностью и/или чередованием	Л5.2
RDIMM / UDIMM	Registered / Unbuffered DIMM	Серверная память с буферным регистром / десктопная	Л2
RISC	Reduced Instruction Set Computer	Подход с малым числом простых команд (ARM, RISC-V)	Л2
ROI	Return on Investment	Окупаемость вложений	Л2

Термин	Расшифровка	Кратко	Где
Runtime	—	Среда выполнения программы (Node.js, JRE)	Л1
SAN	Storage Area Network	Сеть хранения с блочным доступом	Л5.2
SAS	Serial Attached SCSI	Серверный дисковый интерфейс, полный дуплекс, до 22,5 Гбит/с	Л5.1
SATA	Serial ATA	Массовый дисковый интерфейс, полудуплекс, до 6 Гбит/с	Л5.1
SCSI	Small Computer System Interface	Дисковый протокол SAS и SAN	Л5.1
SDN	Software Defined Networking	Программно-определяемая сеть	Л1
SDS	Software Defined Storage	Программно-определяемое хранилище	Л1, Л5.2
SoC	System on a Chip	Вся система на одном кристалле (Apple M, NVIDIA DGX Spark)	Л2
SSD	Solid-State Drive	Твердотельный накопитель на NAND	Л5.1
Target / Initiator	—	Целевое устройство (диск, массив) / источник запросов (контроллер, сервер)	Л5.1, Л5.2
TCO	Total Cost of Ownership	Полная стоимость владения	Л2
TDP	Thermal Design Power	Тепло, которое должна рассеивать система охлаждения	Л2
Tiering	—	Ярусное хранение: горячие данные на SSD, холодные на HDD	Л1, Л5.1
TRIM	—	Команда ОС диску SSD об удалённых блоках	Л5.1

Термин	Расшифровка	Кратко	Где
U (юнит)	Rack Unit	Единица высоты в стойке, 44,45 мм	Л2
VLIW	Very Long Instruction Word	Одна длинная команда с несколькими операциями (Эльбрус)	Л2
VRAM	Video RAM	Память видеокарты	Л2
XOR	Exclusive OR	Исключающее ИЛИ; считает чётность в RAID 5	Л5.2
Макро- / микрокоманда (μop)	—	Инструкция ISA / элементарное действие внутри процессора	Л2
Северный / южный мост	Northbridge / Southbridge	Чипы платы для быстрых / медленных устройств	Л2
Фабрика	Fabric	Сеть коммутаторов SAN	Л5.2
Экспандер	SAS Expander	«Коммутатор» SAS: много дисков на один порт	Л5.1